AI Engineer Transition — Phase 1 Study Notes

Python & Tooling, for a Java/Spring backend engineer. Covers lessons 1.1–1.9. Prepared as a reference before starting Phase 2 (LLM fundamentals).

1.1 — Install Python + uv, run first script

DONE

Key ideas

- uv replaces SDKMAN + Maven together: installs Python versions, manages projects, runs scripts.
- `pyproject.toml` is the Python equivalent of `pom.xml`.
- `uv init` can create either a flat script layout or a packaged layout (`src/<pkg>/__init__.py` + `[project.scripts]`) depending on uv version — check which you got before following file-path instructions.
- `if __name__ == "__main__":` — every top-level line in a Python file runs on *import*, not just when run directly. The guard means "only run this when the file IS the program", the closest thing Python has to `public static void main`.

Java/Spring analogy

Java / Spring	Python
`javac + classpath run`	`uv run <file>.py`
`Main-Class in JAR manifest`	`[project.scripts] entry point`
`pom.xml`	`pyproject.toml`

Core snippet

```
if __name__ == "__main__":
    main()

# running directly  -> __name__ == "__main__"  -> main() runs
# imported elsewhere -> __name__ == ""  -> main() does NOT run
```

Takeaway: A [project.scripts] entry point (uv run hello-ai) imports the package and calls main() explicitly, so it never needs the __main__ guard — only a file run directly as a script does.

1.2 — Python syntax for Java devs

DONE

Key ideas

- No braces/semicolons — indentation is the block structure.
- Type hints (`-> float`) are documentation only at this stage, not enforced (changes in 1.8 with Pydantic).
- Comprehensions replace most `.stream().filter().map()` chains.
- Default parameter values replace Java method overloading — one function signature covers every call shape; Python has no overloading at all.

Java/Spring analogy

Java / Spring	Python
`new ArrayList<>()`	`[]`
`new HashMap<>()`	`{}`
`new HashSet<>()`	`set()`
`.stream().filter(c).map(f).toList()`	`[f(x) for x in lst if c]`
`map.getOrDefault(k, 0)`	`d.get(k, 0)`
`Comparator.comparing(Trade::notional)`	`key=notional`
`Overloaded method signatures`	`def f(x, y=default):`

Core snippet

```
# list comprehension
[t for t in trades if t["side"] == "BUY" and notional(t) > min_notional]

# set comprehension
{t["symbol"] for t in trades}

# default parameter instead of method overloading
def buys_over(trades, min_notional=1000.0, side="BUY"): ...
```

Takeaway: One function with default parameters covers what Java needs several overloaded methods for.

1.3 — Classes, dataclasses, type hints

DONE

Key ideas

- `self` is Python's `this`, but must be written explicitly as the first parameter of every instance method.
- `@dataclass` auto-generates `__init__`, `__repr__` (toString), and `__eq__` — close to a Java `record`.
- Critical gap: type hints on a dataclass are NOT checked at runtime. `Trade("AAPL","BUY","ten",190.5)` runs fine with `qty` silently holding a string.
- No compiler gate exists in Python — this is what Pydantic (1.8) exists to fix.

Java/Spring analogy

Java / Spring	Python
`record Trade(String symbol, ...)`	`@dataclass class Trade: ...`
`Compile-time type checking (javac)`	`No built-in equivalent — hints are metadata only`

Core snippet

```
@dataclass
class Trade:
    symbol: str
    side: str
    qty: int
    price: float

    def notional(self) -> float:
        return self.qty * self.price
```

Takeaway: Java's type safety comes from static compilation (javac refuses bad code). Python has no equivalent gate — dataclass hints are purely advisory until Pydantic enforces them.

1.4 — Modules, venv, project structure

DONE

Key ideas

- A module = one .py file. A package = a folder containing `__init__.py` (like a Java package directory).
- `.venv` is a private interpreter + isolated `site-packages` per project — like each Maven project getting its own private `~/.m2`, preventing dependency-version clashes across projects.
- `from tradelib.models import Trade` is package-qualified because Python resolves imports against the import path (like the classpath) — `models.py` only exists inside the `tradelib` namespace, so you must name that package to reach it.
- `uv add <pkg>` = adding a Maven dependency + running install, in one step.

Core snippet

```
hello-ai/
■■■ pyproject.toml
■■■ src/
■   ■■■ tradelib/
■       ■■■ __init__.py
■       ■■■ models.py      # Trade dataclass
■       ■■■ analysis.py    # imports: from tradelib.models import Trade
■■■ main.py               # entry point
```

Takeaway: src// layout plus pyproject.toml's default src-discovery puts the package on the import path automatically — no manual PYTHONPATH wrangling needed.

1.5 — Files, JSON, HTTP requests

DONE

Key ideas

- `with open(path) as f:` is Python's try-with-resources — file auto-closes at block end.
- `json.dump(obj, f, indent=2)`: 1st arg = data to serialize, 2nd arg = the open file handle to write into, `indent=2` = pretty-print (like Jackson's `writerWithDefaultPrettyPrinter()`). `dump` writes to a file; `dumps` (with an s) returns a string instead.
- `dataclasses.asdict()` converts a dataclass to a plain dict before serializing — Python doesn't auto-serialize custom objects the way Jackson can.
- `Trade(**d)` unpacks a dict into keyword constructor args in one call.
- `requests` (3rd-party) replaces raw `urllib`, similar to choosing RestTemplate/WebClient over raw HttpURLConnection.
- **Uncaught exceptions behave like Java unchecked exceptions:** `resp.raise_for_status()` raises `requests.exceptions.HTTPError` (≈ Spring's `HttpClientErrorException`) on 4xx/5xx; if uncaught, it propagates and crashes the program with a traceback — no checked-exception mechanism exists in Python at all.

Core snippet

```
resp = requests.get(url)
resp.raise_for_status()   # raises HTTPError on 4xx/5xx, uncaught -> crash
data = resp.json()
```

Takeaway: A real bug caught during this lesson: positional dataclass construction binds by field ORDER, not by label shown in repr — a mismatched field order silently corrupts data even though nothing crashes and equality checks can still report True.

1.6 — Async/await basics

DONE

Key ideas

- Python's async/await is single-threaded cooperative concurrency — one thread, an event loop. Not like Java thread pools/CompletableFuture running on multiple OS threads; closer to a single-threaded Netty/reactor event loop. Good for overlapping I/O wait time, not CPU parallelism.
- `async def foo():` defines a coroutine function. Calling `foo()` does NOT run it — it returns an unstarted coroutine object (like a CompletableFuture that hasn't been submitted). Nothing inside runs until something *drives* it forward.
- `await` or `asyncio.gather(...)` is what actually starts/advances a coroutine. Only await the specific operations that need to suspend (I/O) — e.g. `resp.json()` needs await (body still streaming in) but `resp.raise_for_status()` does not (status code already in memory).
- `asyncio.gather(a(), b(), c())` ≈ `CompletableFuture.allOf(...)` — runs several coroutines concurrently.

Core snippet

```
tasks = [fetch_repo(session, r) for r in repos]   # 3 unstarted coroutine objects
results = await asyncio.gather(*tasks)            # NOW they all start, concurrently
```

Takeaway: Measured result: sequential 3 HTTP calls ≈0.15s vs concurrent ≈0.04s — roughly 4x faster by overlapping network wait time, not by using multiple CPU cores.

1.7 — pytest basics

OWED

Key ideas

- pytest needs no class, no @Test annotation — a plain function named test_* using Python's built-in assert is the entire API. pytest rewrites assert failures to show both sides automatically, no assertEquals/assertThat needed.
- Test discovery is by filename convention: test_*.py or *_test.py — no XML/registration config.
- pytest.raises(SomeError) ≈ JUnit 5's assertThrows.
- @pytest.fixture ≈ @BeforeEach, but reusable/composable just by naming it as a test parameter.

Java/Spring analogy

Java / Spring	Python
`@Test + assertEquals(a, b)`	`def test_x(): assert a == b`
`assertThrows(E.class, ...)`	`with pytest.raises(E): ...`
`@BeforeEach`	`@pytest.fixture`

■ Skipped during the lesson — not yet verified hands-on. Folded into 1.10's project scope (Part B) so it will be completed together with the capstone, but is not done standalone yet.

1.8 — Pydantic models

DONE

Key ideas

- Closest Python equivalent to Java Bean Validation (@NotNull, @Min, JSR-303) + Jackson deserialization combined. Fixes the exact gap exposed in 1.3.
- Inherit `BaseModel` instead of `@dataclass`. Construction now actually validates types and raises `pydantic.ValidationError` on mismatch.
- Coercion vs rejection: a numeric string like `qty="100"` is converted via `int("100")` and accepted (succeeds); `qty="ten"` fails the same conversion and is rejected. The rule is "try to convert, accept only if conversion succeeds" — not "accept any string".
- `Field(gt=0)` ≈ `@Min(1)` — constraints beyond type.
- `model_dump()` → dict (replaces asdict), `model_dump_json()` → JSON string directly, `model_validate_json(str)` → parse + validate in one call.

Core snippet

```
class Trade(BaseModel):
    symbol: str
    side: str
    qty: int = Field(gt=0)
    price: float = Field(gt=0)
```

Takeaway: Dataclass hints are advisory; Pydantic hints are enforced — this is the real runtime safety net Java developers expect by default, finally present.

1.9 — FastAPI hello world + CRUD

DONE

Key ideas

- FastAPI ≈ @RestController layer of Spring Boot: route decorators instead of @GetMapping, Pydantic models instead of @RequestBody DTOs, free automatic OpenAPI/Swagger docs at /docs.
- `def create_trade(trade: Trade):` — FastAPI automatically parses+validates the raw JSON body into a Trade (via Pydantic's model_validate_json under the hood) and automatically serializes whatever you return back to JSON (via model_dump/model_dump_json). No manual json.loads/json.dump anywhere in the route code.
- Closest single Spring equivalent: @RequestBody + @ResponseBody together (or a @RestController method, which implies both) — Jackson does the same two-way automatic binding based on type hints.
- Route ORDER matters: a literal path like `/trades/summary` must be declared BEFORE a parameterized path like `/trades/{trade_id}`, or the parameterized route swallows it first (tries to parse "summary" as an int id).
- If Trade were a plain dataclass instead of a Pydantic model here: no automatic validation, bad input could crash with an unhelpful 500 instead of a clean structured 422 response.

Java/Spring analogy

Java / Spring	Python
`@RestController`	`module of route functions`
`@GetMapping("/trades")`	`@app.get("/trades")`
`@PostMapping + @RequestBody Trade`	`@app.post("/trades") def f(trade: Trade):`
`@PathVariable`	`function param matching {curly} in path`
`ResponseEntity.status(404)`	`raise HTTPException(status_code=404)`
`embedded Tomcat`	`uvicorn (ASGI server)`

Core snippet

```
@app.post("/trades", status_code=201)
def create_trade(trade: Trade):          # auto-parsed + validated from JSON body
    ...
    return {"id": trade_id, **trade.model_dump()}   # auto-serialized to JSON
```

Takeaway: Verified full CRUD cycle end-to-end with curl: 201 create, list, get-by-id, 404 on missing id, 204 delete, confirmed empty afterward. Working project folder: trade-json.

1.10 — PROJECT — FastAPI trade service with tests

OWED

Key ideas

- Phase 1 capstone. Extends trade-json with: PUT /trades/{id} (full update, 404 if missing), GET /trades/summary (count + net_position via analysis.py — must be declared before the {trade_id} route to avoid route-matching conflicts).
- Folds in the still-owed pytest fundamentals from 1.7 as Part B (3 model-level tests).
- Part C introduces FastAPI's TestClient (via httpx) — calls routes in-process with no running server needed, the Python equivalent of Spring's MockMvc as opposed to a full @SpringBootTest with an embedded server.
- Target: 9 passing tests total (3 pydantic-level + 5 given API tests + 1 self-written), plus a short README for the GitHub push.

Java/Spring analogy

Java / Spring	Python
`MockMvc (in-process route testing)`	`FastAPI TestClient (httpx)`
`@BeforeEach resetting test state`	`def setup_function(): db.clear()`

■ Deferred by choice to move faster into Phase 2 — full spec was given and is ready to pick up any time by saying "do 1.10". Required before 2.7 (Phase 2 project) and later agent-building lessons get materially harder to debug without this practice.

Phase 1 status summary

Lesson	Topic	Status
1.1	uv, Python projects, __main__ guard	**Done**
1.2	Comprehensions, default params	**Done**
1.3	Dataclasses, unenforced type hints	**Done**
1.4	Modules, packages, .venv, project layout	**Done**
1.5	JSON I/O, HTTP requests, uncaught exceptions	**Done**
1.6	async/await, cooperative concurrency	**Done**
1.7	pytest basics	**Owed — folded into 1.10**
1.8	Pydantic — real runtime validation	**Done**
1.9	FastAPI CRUD service	**Done**
1.10	PROJECT — tested FastAPI service	**Owed — spec ready**

What Phase 1 built, end to end

- A full package (tradelib) with models, analysis logic, and a FastAPI CRUD service over it.
- Fluency in comprehensions, dataclasses vs Pydantic, async/await, JSON/HTTP I/O.
- Understanding of where Python's safety nets differ from Java's — no compile-time type checking, no checked exceptions, and why Pydantic exists to close the validation gap.
- Running projects correctly with uv instead of bare python/pip.

Open items before this phase is fully closed

- **1.7** — pytest fundamentals not yet verified standalone.

- **1.10** — capstone project (extended CRUD + full test suite) not yet submitted.
- Say `"do 1.10"` or `"checkpoint"` any time to resume exactly here.

Next: Phase 2 — LLM fundamentals (tokens, context window, Ollama + Qwen locally, prompting, embeddings), fully local-first on your RTX 2050 / 16GB setup.